

Manual del Programador Afora Backend

Documentación técnica para desarrolladores que trabajen en el backend de **Afora**.

Índice

- [Introducción](#)
- [Stack tecnológico](#)
- [Estructura del proyecto](#)
- [Configuración del entorno](#)
- [Arquitectura de la aplicación](#)
- [Modelo de datos](#)
- [Autenticación y autorización](#)
- [API REST — Referencia de endpoints](#)
- [WebSockets — Tiempo real](#)
- [DTOs y validación](#)
- [Módulo común y utilidades](#)
- [Seed de datos](#)
- [Migraciones de base de datos](#)
- [Swagger / OpenAPI](#)
- [Scripts npm/yarn disponibles](#)
- [Convenciones y patrones del código](#)
- [Notas importantes y deuda técnica conocida](#)

1. Introducción

Afora Backend es una API REST construida con [NestJS](#) que gestiona recursos escolares: usuarios, aulas, cursos, materias y clases (asignaciones docente–aula–curso–materia). Incluye autenticación JWT, documentación Swagger y actualizaciones en tiempo real de aulas vía WebSockets (Socket.IO).

Propósito del sistema: centralizar la administración de espacios físicos (aulas), la organización académica (cursos, materias) y la programación de clases, con soporte para consultas paginadas y notificaciones en vivo cuando cambia el estado de las aulas.

Prefijo global de la API: `/apie`

Puerto por defecto: `3000`

API en producción: <https://afora-backend.onrender.com>

Documentación interactiva en desarrollo: <http://localhost:3000/docs>

Documentación interactiva en producción: <https://afora-backend.onrender.com/docs>

2. Stack tecnológico

Tecnología	Versión / Uso
NestJS	11.x — framework principal

TypeScript	5.7.x
TypeORM	0.3.x — ORM sobre PostgreSQL
PostgreSQL	17 (Docker)
Passport + JWT	Autenticación stateless
bcrypt	Hash de contraseñas
class-validator / class-transformer	Validación de DTOs
Swagger (@nestjs/swagger)	Documentación OpenAPI
Socket.IO	WebSockets para aulas
cookie-parser	JWT en cookie httpOnly
Docker Compose	Base de datos local

3. Estructura del proyecto

```
afora-backend/
├── src/
│   ├── main.ts                # Bootstrap: pipes, CORS, Swagger, cookies
│   ├── app.module.ts          # Módulo raíz e integración TypeORM
│   ├── auth/                  # Usuarios, JWT, guards, decoradores
│   ├── aula/                  # CRUD de aulas
│   ├── aula-socket/           # Gateway WebSocket de aulas
│   ├── clase/                 # Asignaciones docente-aula-curso-materia
│   ├── curso/                 # CRUD de cursos
│   ├── materia/               # CRUD de materias
│   ├── common/                # DTOs y utilidades compartidas
│   ├── seed/                  # Poblado inicial de la BD
│   └── migrations/            # Migraciones TypeORM
├── test/                      # Tests e2e
├── docs/                      # Documentación (este manual)
├── docker-compose.yaml        # PostgreSQL local
├── typeorm.config.ts          # DataSource para CLI de migraciones
├── .env.template              # Plantilla de variables de entorno
└── package.json
```

Módulos NestJS y dependencias

```
AppModule
├── ConfigModule (global)
├── TypeOrmModule (PostgreSQL)
├── AuthModule ──────────────────> exporta JWT, Passport, AuthService
├── CommonModule
├── AulaModule <←forwardRef→> AulaSocketModule
```

```
|— CursoModule
|— MateriaModule
|— ClaseModule
|— SeedModule (usa Auth, Aula, Curso, Materia services)
```

4. Configuración del entorno

4.1 Variables de entorno

Copiar `.env.template` a `.env`:

Variable	Descripción	Ejemplo
STAGE	Entorno (<code>dev</code> / <code>prod</code>)	<code>dev</code>
DB_HOST	Host PostgreSQL	<code>localhost</code>
DB_PORT	Puerto PostgreSQL	<code>5432</code>
DB_DATABASE	Nombre de la BD	<code>organimapDB</code>
DB_USERNAME	Usuario PostgreSQL	<code>postgres</code>
DB_PASSWORD	Contraseña PostgreSQL	<code>MySecretPassword</code>
PORT	Puerto HTTP del servidor	<code>3000</code>
HOST_API	Host público de la API	<code>localhost:3000/api</code>
JWT_SECRET	Secreto para firmar tokens JWT	<code>MySecret</code>

4.2 Arranque local

```
# 1. Instalar dependencias
yarn

# 2. Configurar .env (ver arriba)

# 3. Levantar PostgreSQL
docker compose up -d

# 4. Iniciar en modo desarrollo
yarn start:dev
```

4.3 Docker Compose

El servicio `db` expone PostgreSQL 17 en el puerto `5432`. Los datos persisten en `./postgres`.

5. Arquitectura de la aplicación

5.1 Bootstrap (src/main.ts)

Al iniciar la aplicación se configura:

- Prefijo global:** `api` → todas las rutas son `/api/...`
- CORS:** habilitado globalmente (`enableCors()`)
- Cookies:** middleware `cookie-parser`
- ValidationPipe global:**
 - `whitelist: true` — elimina propiedades no declaradas en el DTO
 - `forbidNonWhitelisted: true` — rechaza peticiones con campos extra
- Swagger:** disponible en `/docs`

5.2 Capas por módulo

Cada dominio sigue el patrón estándar de NestJS:

Capa	Responsabilidad
Controller	Rutas HTTP, validación de entrada, respuestas HTTP
Service	Lógica de negocio, acceso a repositorios TypeORM
Entity	Mapeo objeto-relacional (tablas PostgreSQL)
DTO	Contratos de entrada con validación <code>class-validator</code>
Module	Registro de providers, imports y exports

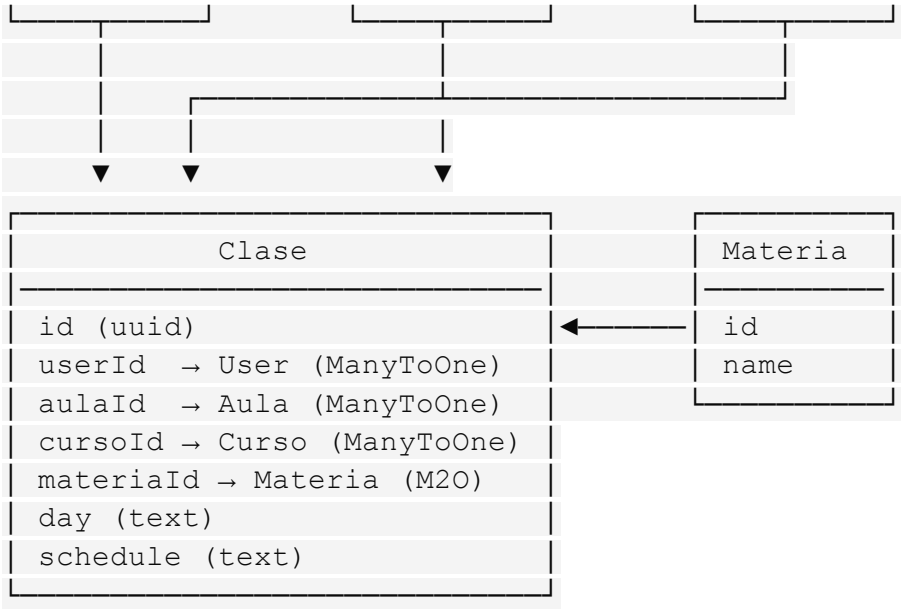
5.3 TypeORM (src/app.module.ts)

```
TypeOrmModule.forRootAsync({
  type: 'postgres',
  autoLoadEntities: true,
  synchronize: true, // Solo desarrollo – ver sección 17
  ssl: true,
  extra: { ssl: { rejectUnauthorized: false } },
})
```

6. Modelo de datos

6.1 Diagrama de relaciones

User		Aula		Curso
id (uuid)		id		id
name		name		name
password		capacity		shift
roles[]		state		numberOf
isActive		desc?		Students



6.2 Entidades

User (`src/auth/entities/user.entity.ts`)

Campo	Tipo	Notas
<code>id</code>	UUID	PK autogenerada
<code>name</code>	text	Único
<code>password</code>	text	Hash bcrypt; <code>select: false</code> por defecto
<code>roles</code>	text[]	Array PostgreSQL de roles
<code>isActive</code>	boolean	Default <code>true</code> ; usuarios inactivos no pueden autenticarse
<code>clase</code>	Clase[]	Relación OneToMany

Aula (`src/aula/entities/aula.entity.ts`)

Campo	Tipo	Notas
<code>id</code>	UUID	PK
<code>name</code>	text	Único
<code>description</code>	text	Opcional
<code>capacity</code>	int	Capacidad de personas

state	enum	available, maintenance, busy
clase	Clase[]	OneToMany

Enum State (src/aula/interfaces/state-values.ts):

```
enum State {  
  AVAILABLE = 'available',  
  MAINTENANCE = 'maintenance',  
  BUSY = 'busy',  
}
```

Curso (src/curso/entities/curso.entity.ts)

Campo	Tipo	Notas
id	UUID	PK
name	text	Único (ej. "7mo 2da")
shift	text	Turno: mañana, tarde, vespertino
numberOfStudents	int	Cantidad de alumnos
clase	Clase[]	OneToMany

Materia (src/materia/entities/materia.entity.ts)

Campo	Tipo	Notas
id	UUID	PK
name	text	Único
clase	Clase[]	OneToMany

Clase (src/clase/entities/clase.entity.ts)

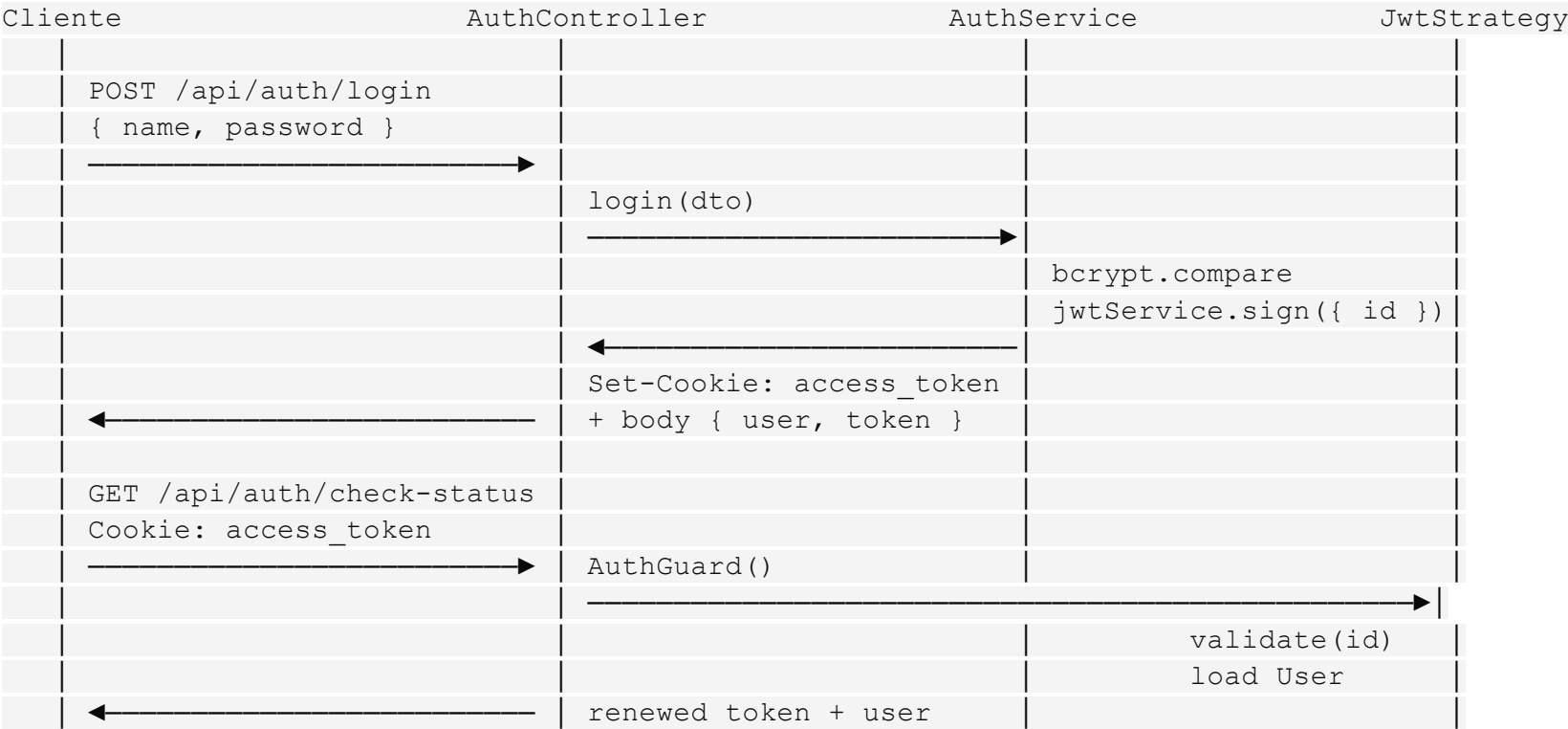
Entidad central que **vincula** docente, aula, curso y materia en un horario.

Campo	Tipo	Notas
id	UUID	PK

user	User	ManyToOne, eager: true
aula	Aula	ManyToOne, eager: true
curso	Curso	ManyToOne, eager: true
materia	Materia	ManyToOne, eager: true
day	text	Día de la semana (ej. "Lunes")
schedule	text	Formato HH:MM-HH:MM (24h)

7. Autenticación y autorización

7.1 Flujo de autenticación



7.2 Extracción del JWT

JwtStrategy acepta el token desde **dos fuentes** (en orden):

Cookie access_token (httpOnly)
Header Authorization: Bearer <token>

7.3 Configuración del token

Secreto: variable JWT_SECRET
Expiración: 2 horas (JwtModule.signOptions.expiresIn: '2h')
Cookie en login/register/check-status:
httpOnly: true
secure: true solo si STAGE=prod
sameSite: 'lax'
Expira en 2 horas

7.4 Roles del sistema

Definidos en `ValidRoles` (`src/auth/interfaces/valid-roles.ts`):

Rol	Valor
Admin	<code>admin</code>
Usuario	<code>user</code>
Supervisor	<code>supervisor</code>

7.5 Decoradores de autorización

`@Auth(...roles)` — Composición de guards

```
// src/auth/decorators/auth.decorator.ts
@Auth(ValidRoles.admin, ValidRoles.user)
findAll() { ... }
```

Aplica:

- `@RoleProtected(...roles)` — metadata en el handler
- `AuthGuard()` — valida JWT (Passport)
- `UserRoleGuard` — comprueba que el usuario tenga al menos uno de los roles requeridos

Si no se pasan roles a `@Auth()`, el guard de roles permite el acceso a cualquier usuario autenticado.

`@GetUser()` — Usuario autenticado en el handler

```
checkAuthStatus(@GetUser() user: User) { ... }

// Con campo específico:
someMethod(@GetUser('id') userId: string) { ... }
```

7.6 Validación de contraseñas

Tanto registro como login exigen:

- Longitud: 6–50 caracteres
- Al menos una mayúscula, una minúscula y un número (o carácter especial)

Regex: `/ (? : ( ? = . * \d ) | ( ? = . * \W + ) ) ( ? ! [ . \n ] ) ( ? = . * [ A - Z ] ) ( ? = . * [ a - z ] ) . * $ /`

8. API REST — Referencia de endpoints

Base URL: `http://localhost:3000/api`

8.1 Auth — `/auth`

POST `/auth/register`

Body (`CreateAuthDto`):

```
{
  "name": "Juan Pérez",
  "password": "MiPass123",
```



```
  "roles": ["user"]
}
```

Respuesta 201: usuario creado (sin password) + token + cookie.

POST /auth/login

Body (LoginUserDto):

```
{
  "name": "Juan Pérez",
  "password": "MiPass123"
}
```

Respuesta 200: { user, token } + cookie.

GET /auth/user

Query params (PaginationDto):

Param	Tipo	Descripción
limit	number	Tamaño de página (default 10)
offset	number	Desplazamiento (default 0)
query	string	Búsqueda por nombre (ILIKE)
roles	string[]	Filtro por roles (ej. roles=admin,user)

Respuesta:

```
{
  "total": 25,
  "pages": 3,
  "users": [ ... ]
}
```

8.2 Aulas — /aula

Método	Ruta	Descripción
POST	/aula	Crear aula + broadcast WebSocket
GET	/aula	Listar paginado
GET	/aula/:id	Obtener por UUID
PATCH	/aula/:id	Actualizar + broadcast
DELETE	/aula/:id	Eliminar + broadcast

Body crear (CreateAulaDto):

```
{
  "name": "Laboratorio 1",
  "description": "Sala de computación",
  "capacity": 30,
  "state": "available"
}
```

Query params en listado:

Param	Descripción
limit, offset	Paginación
query	Filtro por nombre (ILIKE)
state	available, maintenance, busy

Respuesta listado:

```
{
  "total": 4,
  "pages": 1,
  "aulas": [ ... ]
}
```

8.3 Cursos — /curso

Método	Ruta	Descripción
POST	/curso	Crear curso
GET	/curso	Listar paginado (filtro query por nombre)
GET	/curso/:id	Obtener por UUID
PATCH	/curso/:id	Actualizar
DELETE	/curso/:id	Eliminar

Body crear (CreateCursoDto):

```
{
  "name": "7mo 2da",
  "shift": "vespertino",
  "numberOfStudents": 33
}
```

8.4 Materias — /materia

Método	Ruta	Descripción
--------	------	-------------

POST	/materia	Crear materia
GET	/materia	Listar paginado
GET	/materia/:id	Obtener por UUID
PATCH	/materia/:id	Actualizar
DELETE	/materia/:id	Eliminar

Body crear (CreateMateriaDto):

```
{
  "name": "Programación Full Stack"
}
```

8.5 Clases — /clase

Una **Clase** es la asignación de un docente a dictar una materia para un curso en un aula, en un día y horario concretos.

Método	Ruta	Descripción
POST	/clase	Crear asignación
GET	/clase	Listar todas (sin paginación)
GET	/clase/:id	Obtener por UUID
PATCH	/clase/:id	Actualizar
DELETE	/clase/:id	Eliminar

Body crear (CreateClaseDto):

```
{
  "userId": "uuid-del-docente",
  "aulaId": "uuid-del-aula",
  "cursoId": "uuid-del-curso",
  "materiaId": "uuid-de-la-materia",
  "day": "Lunes",
  "schedule": "17:45-19:45"
}
```

Reglas de negocio en creación/actualización:

- El `userId` debe existir y tener el rol `'user'` en su array `roles`
- `aulaId`, `cursoId` y `materiaId` deben existir
- `schedule` debe cumplir el formato `HH:MM-HH:MM` (24 horas)

8.6 Seed — /seed

Método	Ruta	Descripción
GET	/seed	Borra tablas y repuebla datos iniciales

Endpoint **sin autenticación**. Usar solo en desarrollo.

8.7 Códigos de respuesta HTTP comunes

Código	Significado
200	OK
201	Recurso creado
400	Validación fallida o regla de negocio
401	Credenciales o token inválidos
403	Rol insuficiente
404	Recurso no encontrado
500	Error interno (ver logs del servidor)

Errores de unicidad PostgreSQL (23505) se traducen a 400 `Bad Request` con el detalle de la BD.

9. WebSockets — Tiempo real

9.1 Gateway

Archivo: `src/aula-socket/aula-socket.gateway.ts`

Transporte: Socket.IO
CORS WebSocket: `origin: '*'`
Namespace: default (/)

9.2 Eventos del cliente → servidor

Evento emitido	Payload	Respuesta del servidor
<code>findAllAulaSocket</code>	<code>PaginationDto</code> (opcional)	Emite <code>aulasUpdated</code> al cliente que pidió

<code>findOneAulaSocket</code>	<code>string</code> (UUID del aula)	Emite <code>aulaUpdated</code> al cliente
--------------------------------	-------------------------------------	---

9.3 Eventos del servidor → clientes

Evento	Cuándo se emite	Payload
<code>aulasUpdated</code>	Tras CRUD de aula (REST) o petición socket	<code>{ total, pages, aulas }</code>
<code>aulaUpdated</code>	Tras consulta individual por socket	Objeto <code>Aula</code> completo

9.4 Integración REST + WebSocket

`AulaController` llama a `aulaSocketGateway.broadcastAulas()` después de `create`, `update` y `remove`, notificando a **todos** los clientes conectados.

9.5 Ejemplo cliente (JavaScript)

```
import { io } from 'socket.io-client';

const socket = io('http://localhost:3000');

socket.emit('findAllAulaSocket', { limit: 10, offset: 0 });

socket.on('aulasUpdated', (data) => {
  console.log('Aulas actualizadas:', data);
});

socket.on('aulaUpdated', (aula) => {
  console.log('Aula individual:', aula);
});
```

9.6 Dependencia circular

`AulaModule` y `AulaSocketModule` se referencian mutuamente con `forwardRef()`. Esto permite que el controller REST inyecte el gateway para broadcasts.

10. DTOs y validación

10.1 PaginationDto (compartido)

Usado en listados de usuarios y aulas.

```
class PaginationDto {
  limit?: number; // @IsPositive()
  offset?: number;
  query?: string; // Búsqueda por nombre
  state?: State; // Solo aulas: available | maintenance | busy
  roles?: string[]; // Solo usuarios; acepta "admin,user" como string
}
```

El transform `@Transform` en `roles` convierte `"admin,user"` en `['admin', 'user']`.

10.2 DTOs de actualización

Todos los módulos usan `PartialType(CreateXxxDto)` de `@nestjs/swagger` para PATCH:

`UpdateAuthDto`, `UpdateAulaDto`, `UpdateCursoDto`, `UpdateMateriaDto`, `UpdateClaseDto`

Solo los campos enviados se actualizan (`preload` + `save` en TypeORM).

10.3 ValidationPipe global

Cualquier propiedad no declarada en el DTO provoca `400 Bad Request` automáticamente.

11. Módulo común y utilidades

11.1 `handleDbError (src/common/utils/handle-errors.ts)`

Función centralizada para errores de base de datos:

```
function handleDbError(error: any): never {
  if (error?.code === '23505') {
    throw new BadRequestException(error.detail); // Violación UNIQUE
  }
  logger.error(error);
  throw new InternalServerErrorException('Unexpected error, check server logs');
}
```

Usada en todos los services al persistir datos.

11.2 CommonModule

Módulo vacío de registro; los DTOs y utilidades se importan directamente por path relativo.

12. Seed de datos

12.1 Ejecución

```
curl http://localhost:3000/api/seed
```

12.2 Proceso (`SeedService.runSeed`)

- Borrado:** elimina todos los registros de aulas, cursos, materias y usuarios (en ese orden)
- Inserción:** crea usuarios, aulas, cursos y materias desde `src/seed/data/seed-data.ts`

12.3 Datos iniciales incluidos

Recurs o	Cantidad	Ejemplos
Usuarios	3	Santiago Robles (admin), Bianca Docente (user), Secretaria General (admin)
Aulas	4	Lab Informática, Aula 102, Taller Electrónica, Biblioteca
Cursos	3	7mo 2da, 6to 1ra, 5to 3ra

Materias	5	Programación, BD, SO, Electrónica, Matemática
----------	---	---

Los datos de mapas en `seed-data.ts` están definidos pero **no se insertan** actualmente.

13. Migraciones de base de datos

Configuración CLI en `typeorm.config.ts`. Entidades y migraciones se resuelven por glob:
`src/**/*.entity{.ts,.js}`
`src/migrations/*{.ts,.js}`

Comandos

Comando	Descripción
<code>yarn migration:generate</code>	Genera migración (dev)
<code>yarn migration:run-dev</code>	Ejecuta migraciones (dev)
<code>yarn migration:revert-dev</code>	Revierte última migración (dev)
<code>yarn migration:generate-prod</code>	Genera migración (prod)
<code>yarn migration:run-prod</code>	Ejecuta migraciones (prod)
<code>yarn migration:revert-prod</code>	Revierte última migración (prod)

En desarrollo, `synchronize: true` en `AppModule` crea/altera tablas automáticamente. Las migraciones son el camino recomendado para producción.

14. Swagger / OpenAPI

URL: `http://localhost:3000/docs`
Título: Afora RESTFul API
Entidades y DTOs anotados con `@ApiProperty` y `@ApiTags` en controllers

Swagger refleja la estructura de request/response.

15. Scripts npm/yarn disponibles

Script	Comando	Uso
--------	---------	-----

<code>start:dev</code>	<code>nest start --watch</code>	Desarrollo con hot-reload
<code>start:debug</code>	<code>nest start --debug --watch</code>	Debug con inspector
<code>start:prod</code>	<code>node dist/main</code>	Producción
<code>build</code>	<code>nest build</code>	Compilar a <code>dist/</code>
<code>lint</code>	ESLint + fix	Calidad de código
<code>format</code>	Prettier	Formateo
<code>test</code>	Jest unitario	Tests en <code>src/</code>
<code>test:e2e</code>	Jest e2e	Tests en <code>test/</code>
<code>test:cov</code>	Cobertura	Reporte de coverage

16. Convenciones y patrones del código

16.1 Nomenclatura

- Archivos:** kebab-case (`create-aula.dto.ts`, `jwt-strategy.ts`)
- Clases:** PascalCase (`AulaService`, `CreateAulaDto`)
- Endpoints:** singular en español/inglés mixto (`/aula`, `/curso`, `/materia`, `/clase`)
- IDs:** UUID v4 (`ParseUUIDPipe` en params de ruta)

16.2 Patrón CRUD estándar en services

```
async create(dto) {
  const entity = repository.create(dto);
  await repository.save(entity);
  return this.findOne(entity.id); // Devuelve con relaciones
}

async findAll(paginationDto) {
  const [items, total] = await repository.findAndCount({ take, skip, where, relations });
  return { total, pages: Math.ceil(total / limit), items };
}

async update(id, dto) {
  const entity = await repository.preload({ id, ...dto });
  if (!entity) throw new NotFoundException(...);
  await repository.save(entity);
  return this.findOne(id);
}

async remove(id) {
  const entity = await this.findOne(id);
  await repository.remove(entity);
}
```



```
return 'DELETE ...';
}
```

16.3 Respuestas paginadas

Formato consistente:

```
{
  "total": <número total de registros>,
  "pages": <total de páginas>,
  "<recurso>s": [ ... ]
}
```

La clave del array varía: `users`, `aulas`, `cursos`, `materias`.

16.4 Relaciones eager en Clase

La entidad `Clase` carga automáticamente `user`, `aula`, `curso` y `materia` en cada consulta (`eager: true`). Esto simplifica las respuestas pero puede impactar rendimiento en listados grandes.

16.5 Añadir un nuevo módulo de dominio

- Generar con Nest CLI: `nest g res -no-spec`,
- Crear entity en `entities/recurso.entity.ts`
- Crear DTOs en `dto/`
- Registrar entity en `TypeOrmModule.forFeature([Recurso])`
- Importar módulo en `AppModule`
- Añadir `@ApiTags` y `@ApiResponse` al controller
- Usar `handleDbError` en el service
- Si requiere auth, aplicar `@Auth(...roles)` en handlers

17. Notas importantes

17.1 Seguridad

Aspecto	Estado actual	Recomendación
Endpoint <code>/seed</code>	Público, borra datos	Deshabilitar o proteger en producción
CORS	Abierto (<code>enableCors()</code>)	Restringir <code>origin</code> en producción
<code>synchronize: true</code>	Activo en <code>AppModule</code>	Desactivar en prod; usar migraciones
SSL en TypeORM	<code>rejectUnauthorized: false</code>	Configurar certificados válidos en prod

Apéndice A — Mapa rápido de archivos clave

Archivo	Función
src/main.ts	Punto de entrada, configuración global
src/app.module.ts	Registro de módulos y TypeORM
src/auth/strategy/jwt-strategy.ts	Validación JWT desde cookie o Bearer
src/auth/decorators/auth.decorator.ts	Guard compuesto Auth + Roles
src/common/dto/pagination.dto.ts	DTO compartido de paginación
src/common/utils/handle-errors.ts	Manejo centralizado de errores BD
src/aula-socket/aula-socket.gateway.ts	WebSocket gateway
src/seed/data/seed-data.ts	Datos iniciales de desarrollo
typeorm.config.ts	Config CLI migraciones